

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. Test Strategy & Plan Sign-Off	7

Document Control

Field	Detail
System Under Test (SUT)	GreenLoom
Team Repo URL	https://github.com/guirong0109/greenloom
Project Board URL	https://greenloom.streamlit.app/

Objective:

The objective of this Quality Assurance Testing Documentation is to ensure that the GreenLoom system:

- (a) Accurately captures operational data (petrol, electricity, wastage)
- (b) Correctly computes carbon emissions and tax liabilities
- (c) Produces reliable AI-driven insights for procurement decisions
- (d) Maintains system stability under real-world usage scenarios
- (e) Prevents critical failures, incorrect financial outputs, and AI hallucinations

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

1.1 In-Scope Core Features

- Petrol receipt extraction
- Electricity data ingestion
- Wastage AI identification
- Carbon emission calculation
- Carbon tax computation
- Procurement AI analysis
- PDF report generation
- Supplier logistics tracking
- Chat-based material extraction

1.2 Out-of-Scope

- External API uptime - Reliability of third-party APIs
- Regulatory validation - Legal certification of carbon calculations
- Mobile app deployment - Native mobile platform testing
- Large-scale enterprise load - More than 10000 concurrent users

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1-5)	Severity (1-5)	Risk Score (LxS)	Mitigation Strategy	Testing Approach
AI hallucinated outputs	4	5	20	Add validation rules + fallback logic	AI boundary testing
Incorrect emission calculation	3	5	15	Unit test formulas thoroughly	Unit testing
Receipt extraction failure	4	4	16	Manual override + retries	Integration testing
API downtime (AI services)	3	5	15	Graceful fallback responses	Fault simulation
Invalid JSON from AI	4	4	16	Regex + validation checks	Edge testing
Database corruption (SQLite)	2	4	8	Backup + validation	Integration testing
Performance lag (large inputs)	3	3	9	Input limits + optimization	Load testing

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
 - Scope: Carbon emission calculation, carbon tax computation, emission factor mapping and data parsing (JSON, regex extraction)
 - Execution: Using Python-based test script to trigger automatically during development and CI pipeline to test each function with predefined inputs and expected outputs
 - Isolation: External dependencies are mocked and no real API calls are made, the test inputs are controlled and deterministic
 - Pass condition: Output matches expected results exactly, no exceptions or runtime errors and all assertions pass successfully
- **Integration Test**
 - Scope: Integration testing validates how multiple components interact together in real workflows
 - Execution: Conducted in a staging environment with real or simulated API calls, end-to-end workflows are triggered using realistic test inputs, logs are monitored for errors or inconsistencies
 - Workflow: Upload petrol receipt, extract litres using AI, convert to emissions, apply carbon tax and generate report
 - Pass condition: Entire workflow completes without failure, data flows correctly between components and outputs are logically consistent and accurate
- **Test Environment (CI/CD Practice):**
 - Local testing: Conducted on developer machines using local database and mocked API responses to fast debugging and validation
 - Staging/CI environment: Cloud-based or hosting testing environment using real APIs and sample datasets to stimulate real-world usage before deployment
 - CI/CD automated pipeline: Triggered on code commits and pull requests including automatic build verification and unit and integration test execution
- **Regression Testing & Pass/Fail Rules:**
 - Execution Phase: Triggered after new feature implementation, bug fixes and system updates
 - Pass/Fail Condition: Test case pass if only all critical test cases succeed and no previously working feature is broken. If any of the criteria is unable to satisfy, it will consider as fail
 - Continuation Rule: If regression tests fail, deployment is blocked and issues must be fixed before proceeding. If regression tests pass, the system proceeds to the next stage
- **Test Data Strategy:**
 - Manual: Sample receipts, controlled carbon scenarios, edge cases
 - Automated: Generated datasets (large-scale simulated inputs, randomised test cases) which includes chat logs and bulk carbon calculations for stress testing and scalability validation

- **Passing Rate Threshold:**

- A minimum of 85% of all test cases that are executed must pass. For the critical tests, all the tests (100%) need to be passed and there is no exception for that. Because critical failure costs huge in production.

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	Requirements	Project	Pass/Failed
Automatic Build	Must compile successfully	GreenLoom	Pass (Fail for GLM server down)
Unit Tests	>= 90% pass	GreenLoom	Pass (Fail for GLM server down)
Code Quality	No critical lint issues	GreenLoom	Pass (Fail for GLM server down)
Test Coverage	>= 80%	GreenLoom	Pass (Fail for GLM server down)

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	100% critical pass	GreenLoom	Pass (Fail for GLM server down)
AI Output Pass Rate	>= 85% valid responses	GreenLoom	Pass (Fail for GLM server down)
Critical Bugs	0 unresolved	GreenLoom	Pass (Fail for GLM server down)
API Performance	< 3 s response time	GreenLoom	Pass (Fail for GLM server down)
Security	No exposed API keys	GreenLoom	Pass (Fail for GLM server down)

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (End-to-End Flow)	Full workflow from receipt upload → carbon report	Upload petrol + electricity + waste → generate report	Accurate carbon dioxide emission and tax report generated	Successful
TC-02	Negative Case (Invalid Input)	Upload corrupted image	Upload invalid file	System will detect invalid file	Successful
TC-03	NFR (Performance)	Large dataset processing	Input large chat + images	Response < 10 s	Successful
TC-04	NFR (Load Test)	Multiple concurrent users	Simulate 50 users	System remains stable	Successful

6. AI Output & Boundary Testing

6.1. Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"Compare virgin vs recycled steel emissions"	Correct EF comparison + calculation	"The price for recycled steel is higher, but the carbon emission for virgin is higher"	Successful
AI-02	"Calculate carbon tax for 15000000 kg steel"	Accurate RM value	"The carbon tax for virgin steel is RM 429750 while for recycled steel is RM 96750"	Successful
AI-03	"Recommend supplier based on carbon"	Logical recommendation	"Supplier with recycled steel is better"	Successful

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	3000 tokens
Input used while testing	Full chat history
Expected Behavior	System processes without crash
Actual Behavior	System processes without crash and able to generate full report
Status	Successful

6.3. Adversarial/Edge Prompt Test

Prompt	Behaviour
"Calculate carbon for XXX)(*!!"	AI should reject or return fallback
System Handling	Detect invalid → return error message

6.4. Hallucinating Handling

The system mitigates hallucination by:

- (a) Restricting AI with database-grounded prompts
- (b) Validating outputs
- (c) Returning fallback message